

CORE CONCEPTS

Use Claude Code features in the SDK

Load project instructions, skills, hooks, and other Claude Code features into your SDK agents.

The Agent SDK is built on the same foundation as Claude Code, which means your SDK agents have access to the same filesystem-based features: project instructions (`CLAUDE.md` and rules), skills, hooks, and more.

When you omit `settingSources`, `query()` reads the same filesystem settings as the Claude Code CLI: user, project, and local settings, `CLAUDE.md` files, and `.claude/` skills, agents, and commands. To run without these, pass `settingSources: []`, which limits the agent to what you configure programmatically. Managed policy settings and the global `~/.claude.json` config are read regardless of this option. See [What settingSources does not control](#).

For a conceptual overview of what each feature does and when to use it, see [Extend Claude Code](#).

Control filesystem settings with settingSources

The setting sources option (`setting_sources` in Python, `settingSources` in TypeScript) controls which filesystem-based settings the SDK loads. Pass an explicit list to opt in to specific sources, or pass an empty array to disable user, project, and local settings.

This example loads both user-level and project-level settings by setting `settingSources` to `["user", "project"]`:

```

from claude_agent_sdk import query, ClaudeAgentOptions, AssistantMessage,
ResultMessage

async for message in query(
    prompt="Help me refactor the auth module",
    options=ClaudeAgentOptions(
        # "user" loads from ~/.claude/, "project" loads from ~/.claude/ in
cwd.

        # Together they give the agent access to CLAUDE.md, skills, hooks, and
        # permissions from both locations.
        setting_sources=["user", "project"],
        allowed_tools=["Read", "Edit", "Bash"],
    ),
):
    if isinstance(message, AssistantMessage):
        for block in message.content:
            if hasattr(block, "text"):
                print(block.text)
    if isinstance(message, ResultMessage) and message.subtype == "success":
        print(f"\nResult: {message.result}")

```

Each source loads settings from a specific location, where `<cwd>` is the working directory you pass via the `cwd` option, or the process's current directory if unset. For the full type definition, see [SettingSource](#) (TypeScript) or [SettingSource](#) (Python).

Source	What it loads	Location
"project"	Project CLAUDE.md, <code>~/.claude/rules/*.md</code> , project skills, project hooks, project <code>settings.json</code>	<code><cwd>/~/.claude/</code> for <code>settings.json</code> and hooks; <code><cwd></code> and every parent directory for CLAUDE.md and rules; <code><cwd></code> and every parent directory up to the repository root for skills
"user"	User CLAUDE.md, <code>~/.claude/rules/*.md</code> , user skills, user settings	<code>~/.claude/</code>
"local"	CLAUDE.local.md, <code>~/.claude/settings.local.json</code>	<code><cwd>/~/.claude/</code> for <code>settings.local.json</code> ; <code><cwd></code> and every parent directory for CLAUDE.local.md

Omitting `settingSources` is equivalent to `["user", "project", "local"]`.

The `cwd` option determines where the SDK looks for project-level inputs. CLAUDE.md and rules load from `<cwd>` and from every parent directory. Skills load from `<cwd>` and from every parent

directory up to the repository root. Project `settings.json` and hooks load only from `<cwd>/.claude/` with no parent-directory fallback.

What settingSources does not control

`settingSources` covers user, project, and local settings. A few inputs are read regardless of its value:

Input	Behavior	To disable
Managed policy settings	Endpoint-managed policy, such as an MDM plist, registry policy, or managed settings file, loads from the host. Server-managed settings are fetched on an eligible configuration when the session authenticates with an organization OAuth login or a directly configured API key	Endpoint policy: remove the managed settings file, plist, or registry policy from the host. Server-managed settings: controlled by your org admin; cannot be disabled from the SDK
<code>~/.claude.json</code> global config	Always read	Relocate with <code>CLAUDE_CONFIG_DIR</code> in <code>env</code>
Auto memory at <code>~/.claude/projects/<project>/memory/</code>	Loaded by default into the system prompt	Set <code>autoMemoryEnabled: false</code> in settings, or <code>CLAUDE_CODE_DISABLE_AUTO_MEMORY=1</code> in <code>env</code>
<u>claude.ai MCP connectors</u>	Loaded when the active authentication method is a claude.ai subscription. Passing <code>mcpServers: {}</code> does not suppress them	Set <code>strictMcpConfig: true</code> , <u>disableClaudeAiConnectors: true</u> in settings, or <code>ENABLE_CLAUDEAI_MCP_SERVERS=false</code> in <code>env</code>

⚠ Do not rely on default `query()` options for multi-tenant isolation. Because the inputs above are read regardless of `settingSources` , an SDK process can pick up host-level configuration and per-directory memory. For multi-tenant deployments, run each tenant in its own filesystem and set `settingSources: []` plus `CLAUDE_CODE_DISABLE_AUTO_MEMORY=1` in `env` . **Server-managed settings** are fetched when the process authenticates with an organization credential; filesystem isolation does not remove them. See **Secure deployment**.

Project instructions (CLAUDE.md and rules)

`CLAUDE.md` files and `.claude/rules/*.md` files give your agent persistent context about your project: coding conventions, build commands, architecture decisions, and instructions. When `settingSources` includes `"project"` (as in the example above), the SDK loads these files into context at session start. The agent then follows your project conventions without you repeating

them in every prompt.

CLAUDE.md load locations

Level	Location	When loaded
Project (root)	<code><cwd>/CLAUDE.md</code> or <code><cwd>/.claude/CLAUDE.md</code>	<code>settingSources</code> includes <code>"project"</code>
Project rules	<code><cwd>/.claude/rules/*.md</code> and <code>.claude/rules/*.md</code> in every parent directory	<code>settingSources</code> includes <code>"project"</code>
Project (parent dirs)	<code>CLAUDE.md</code> files in directories above <code>cwd</code>	<code>settingSources</code> includes <code>"project"</code> , loaded at session start
Project (child dirs)	<code>CLAUDE.md</code> files in subdirectories of <code>cwd</code>	<code>settingSources</code> includes <code>"project"</code> , loaded on demand when the agent reads a file in that subtree
Local	<code><cwd>/CLAUDE.local.md</code> and <code>CLAUDE.local.md</code> in every parent directory	<code>settingSources</code> includes <code>"local"</code>
User	<code>~/.claude/CLAUDE.md</code>	<code>settingSources</code> includes <code>"user"</code>
User rules	<code>~/.claude/rules/*.md</code>	<code>settingSources</code> includes <code>"user"</code>

All levels are additive: if both project and user CLAUDE.md files exist, the agent sees both. There is no hard precedence rule between levels; if instructions conflict, the outcome depends on how Claude interprets them. Write non-conflicting rules, or state precedence explicitly in the more specific file (“These project instructions override any conflicting user-level defaults”).

💡 You can also inject context directly via `systemPrompt` without using CLAUDE.md files. See [Modify system prompts](#). Use CLAUDE.md when you want the same context shared between interactive Claude Code sessions and your SDK agents.

For how to structure and organize CLAUDE.md content, see [Manage Claude’s memory](#).

Skills

Skills are markdown files that give your agent specialized knowledge and invocable workflows. Unlike `CLAUDE.md` (which loads every session), skills load on demand. The agent receives skill descriptions at startup and loads the full content when relevant.

Skills are discovered from the filesystem through `settingSources` . When the `skills` option on `query()` is omitted, discovered user and project skills are enabled and the Skill tool is available, matching CLI behavior. To control which skills are enabled, pass `skills` as `"all"` , a list of skill names, or `[]` to disable all. When `skills` is set, the SDK adds the Skill tool to `allowedTools` automatically. If you also pass an explicit `tools` list, include `"Skill"` in that list so Claude can invoke skills.

```
from claude_agent_sdk import query, ClaudeAgentOptions, ResultMessage

# Skills in .claude/skills/ are discovered automatically
# when settingSources includes "project"
async for message in query(
    prompt="Review this PR using our code review checklist",
    options=ClaudeAgentOptions(
        setting_sources=["user", "project"],
        skills="all",
        allowed_tools=["Read", "Grep", "Glob"],
    ),
):
    if isinstance(message, ResultMessage) and message.subtype == "success":
        print(message.result)
```

❗ Skills must be created as filesystem artifacts (`.claude/skills/<name>/SKILL.md`). The SDK does not have a programmatic API for registering skills. See [Agent Skills in the SDK](#) for full details.

For more on creating and using skills, see [Agent Skills in the SDK](#).

Hooks

The SDK supports two ways to define hooks, and they run side by side:

- **Filesystem hooks:** shell commands defined in `settings.json` , loaded when `settingSources` includes the relevant source. These are the same hooks you'd configure for [Interactive Claude Code sessions](#).
- **Programmatic hooks:** callback functions passed directly to `query()` . These run in your application process and can return structured decisions. See [Control execution with hooks](#).

Both types execute during the same hook lifecycle. If you already have hooks in your project's `.claude/settings.json` and you set `settingSources: ["project"]` , those hooks run

automatically in the SDK with no extra configuration.

Hook callbacks receive the tool input and return a decision dict. Returning `{}` means allow the tool to proceed. To block execution, return a `hookSpecificOutput` object with `permissionDecision:` `"deny"` and a `permissionDecisionReason`. The reason is sent to Claude as the tool result. The top-level `decision` and `reason` fields are deprecated for `PreToolUse`. See the [hooks guide](#) for the full callback signature and return types.

```

from claude_agent_sdk import query, ClaudeAgentOptions, HookMatcher,
ResultMessage

# PreToolUse hook callback. Positional args:
#   input_data: HookInput dict with tool_name, tool_input, hook_event_name
#   tool_use_id: str | None, the ID of the tool call being intercepted
#   context: HookContext, carries session metadata
async def audit_bash(input_data, tool_use_id, context):
    command = input_data.get("tool_input", {}).get("command", "")
    if "rm -rf" in command:
        return {
            "hookSpecificOutput": {
                "hookEventName": "PreToolUse",
                "permissionDecision": "deny",
                "permissionDecisionReason": "Destructive command blocked",
            }
        }
    return {} # Empty dict: allow the tool to proceed

# Filesystem hooks from .claude/settings.json run automatically
# when settingSources loads them. You can also add programmatic hooks:
async for message in query(
    prompt="Refactor the auth module",
    options=ClaudeAgentOptions(
        setting_sources=["project"], # Loads hooks from .claude/settings.json
        hooks={
            "PreToolUse": [
                HookMatcher(matcher="Bash", hooks=[audit_bash]),
            ]
        },
    ),
):
    if isinstance(message, ResultMessage) and message.subtype == "success":
        print(message.result)

```

When to use which hook type

Hook type	Best for
Filesystem (<code>settings.json</code>)	Sharing hooks between CLI and SDK sessions. Supports <code>"command"</code> (shell scripts), <code>"http"</code> (POST to an endpoint), <code>"mcp_tool"</code> (call a connected MCP server's tool), <code>"prompt"</code> (LLM evaluates a prompt), and <code>"agent"</code> (spawns a verifier agent). These fire in the main agent and any subagents it spawns.
Programmatic (callbacks in <code>query()</code>)	Application-specific logic, structured decisions, and in-process integration. These also fire inside subagents. The callback receives <code>agent_id</code> and <code>agent_type</code> to distinguish.

❗ The TypeScript SDK supports additional hook events beyond Python, including `SessionStart` , `SessionEnd` , `TeammateIdle` , and `TaskCompleted` . See the [hooks guide](#) for the full event compatibility table.

For full details on programmatic hooks, see [Control execution with hooks](#). For filesystem hook syntax, see [Hooks](#).

Choose the right feature

The Agent SDK gives you access to several ways to extend your agent's behavior. If you're unsure which to use, this table maps common goals to the right approach.

You want to...	Use	SDK surface
Set project conventions your agent always follows	CLAUDE.md	<code>settingSources: ["project"]</code> loads it automatically
Give the agent reference material it loads when relevant	Skills	<code>settingSources</code> + <code>skills</code> option
Run a reusable workflow (deploy, review, release)	User-invocable skills	<code>settingSources</code> + <code>skills</code> option
Delegate an isolated subtask to a fresh context (research, review)	Subagents	<code>agents</code> parameter + <code>allowedTools: ["Agent"]</code>
Coordinate multiple Claude Code instances with shared task lists and direct inter-agent messaging	Agent teams	Not directly configured via SDK options. Agent teams are a CLI feature where one session acts as the team lead, coordinating work across independent teammates
Run deterministic logic on tool calls (audit, block, transform)	Hooks	<code>hooks</code> parameter with callbacks, or shell scripts loaded via <code>settingSources</code>

You want to...	Use	SDK surface
Give Claude structured tool access to an external service	<u>MCP</u>	<code>mcpServers</code> parameter

💡 **Subagents versus agent teams:** Subagents are ephemeral and isolated: fresh conversation, one task, summary returned to parent. Agent teams coordinate multiple independent Claude Code instances that share a task list and message each other directly. Agent teams are a CLI feature. See What subagents inherit and the agent teams comparison for details.

Every feature you enable adds to your agent’s context window. For per-feature costs and how these features layer together, see Extend Claude Code.

Related resources

- Extend Claude Code: Conceptual overview of all extension features, with comparison tables and context cost analysis
- Skills in the SDK: Full guide to using skills programmatically
- Subagents: Define and invoke subagents for isolated subtasks
- Hooks: Intercept and control agent behavior at key execution points
- Permissions: Control tool access with modes, rules, and callbacks
- System prompts: Inject context without CLAUDE.md files